Contents

1 AI Prompt Engineering **2**
1.1 Prompt Engineering Phases 2
1.1.1 short 2
1.1.2 Prompt Execution 2

2 Prompts **3**
2.1 Prompt 0: configure prompt engineering 3
2.2 Prompt 1: overview of the Parametric Polymorphism 4

3 Methodology and Implementation **5**
3.1 Analysis of the Algorithms 5

Chapter 1

AI Prompt Engineering

For the tecnical accurancy of the output, following the guierdeline of to the project guidelines, the report is generate using **Two-phase prompt strategy**: prompt improvement and prompt execution.

1.1 Prompt Engineering Phases

1.1.1 Prompt Improvement

The first phase is the prompt improvement, where the initial prompt is redefined and improve the given prompt following the project guidelines and a rigorous accademic approach. Every prompt improvement is indipendent from the previous prompt, and the output is generated using the improved prompt.

1.1.2 Prompt Execution

The second phase is the prompt execution, where the improved prompt is used to generate the final output. In this phase, the chat is invited to also give the source reference use to generate the output.

Chapter 2

Prompts

2.1 Prompt 0: configure prompt engineering

Model: Gemini 3.1 Pro extended reasoning

Prompt:

```
System Persona: Act as a Senior Prompt Engineer specializing in
constructing technical prompts for academic writing and code-generation
tasks.

Goal: Given, as input, a task specification (persona, goal,
requirements, writing-style constraints, output structure) for
producing a technical-academic report, produce a single, self-contained
master prompt -- ready to be pasted into a new conversation with a
language model -- which, when executed, will generate that report.

Requirements:

Task 1: Extraction & Preservation. Parse the input specification and
identify its persona/role, goal (including title), all requested tasks
with their respective details, writing-style constraints, and required
output structure. Preserve the full content and intent of the original
specification: do not add requirements, do not omit any, and do not
alter their meaning.

Task 2: Correction & Structuring. Correct any ambiguities, grammatical
errors, or malformed formatting present in the input (e.g., improperly
nested quotation marks). Reorganize the extracted elements into a
clear, logically structured master prompt, using the same section
labels as the input where applicable (e.g., Persona, Goal,
Requirements, Writing Style Constraint, Output Structure).

Writing Style Constraint (STRICT): Do not execute the input
specification and do not produce the report itself. Do not include
```

placeholders, editorial notes, comments, or meta-commentary addressed to the user. Do not use first-person forms ("I," "we") anywhere in the generated master prompt unless they were already required by the input specification's own writing-style constraints.

Output Structure: Return only the generated master prompt, formatted as ready-to-use text, structured under the same section headings as the input (System Persona / Goal / Requirements / Writing Style Constraint / Output Structure), with no additional text before or after it.

Output: Add to the context of the model the persona, formatting constrainghts and the goal of the report.

2.2 Prompt 1: overview of the Parametric Polymorphism

Model: Gemini 3.1 Pro extended reasoning.
Goal: Generate an overview of the Parametric Polymorphism and ad-hoc polymorphism. Giving details of type erasure, compared with generics, and monomorphization. Also compared differences between source code, byte code and binary code.
Prompt:
- Provide an exhaustive theoretical background on parametric polymorphism.
- Define and thoroughly explain Christopher Strachey's distinction between parametric polymorphism and ad-hoc polymorphism.
- Detail the two primary implementation strategies discussed in the literature:
1. Homogeneous translation (type erasure), detailing how it results in one compiled representation for all instantiations.
2. Heterogeneous translation (reification/monomorphization), detailing how it results in separate code generated per instantiated type.
- Analyze type erasure versus reified generics as the core axis of comparison.
- Analyze compile-time versus runtime characteristics as the second axis of comparison, specifically examining when instantiation happens and what type information survives into the compiled binary or bytecode.
- The entire document must be written in professional, compilation-ready LaTeX code.

Chapter 3

Methodology and Implementation

This chapter covers the technical details, algorithms, and design choices.

3.1 Analysis of the Algorithms

You can use sections to divide your chapters into smaller, readable chunks.